

**МОСКОВСКИЙ
ПОЛИТЕХ**

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное автономное
образовательное учреждение высшего образования
«Московский Политехнический Университет»

Факультет информационных технологий
Кафедра информационной безопасности

Отчет по лабораторной работе №4

по дисциплине

«Высоконагруженные системы»

Выполнил:

студент группы 211-331

Скрыпник В.А.

Подпись и дата:

Проверил:

Гневшев А.Ю.

Подпись и дата:

Оглавление

ВВЕДЕНИЕ.....	3
1. АРХИТЕКТУРА СИСТЕМЫ.....	4
1.1. Архитектура системы	4
2. ПРАКТИЧЕСКАЯ ЧАСТЬ.....	6
2.1. Трехзвенная архитектура и микросервисы	6
2.2. Конфигурация Nginx: балансировка и кэширование	6
2.3. Настройка базы данных с репликацией	7
2.4. Асинхронная обработка.....	8
2.5. Симуляция DNS-балансировки	8
2.6. Мониторинг и логирование.....	9
2.7. Резервное копирование и восстановление	10
2.8. Event Sourcing и партиционирование.....	13
2.9. Канареечное развертывание (Canary Deployment)	15
ЗАКЛЮЧЕНИЕ	16
СПИСОК ЛИТЕРАТУРЫ	17

ВВЕДЕНИЕ

Цель работы: спроектировать и реализовать масштабируемую и отказоустойчивую архитектуру высоконагруженной системы с учетом горизонтального масштабирования, микросервисной архитектуры, асинхронной обработки и анализа компромиссов CAP-теоремы.

В соответствии с заданием, на основе ФИО и номера группы был рассчитан индивидуальный вариант.

```
PS C:\Users\basil> python
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from hashlib import sha3_256
... study_group = "211-331" # Укажите номер учебной группы
... fullname = "Скрипник Василий Александрович" # Укажите свои ФИО
... suffix = "Высоконагруженные системы. Лабораторная работа 4"
... msg = f"{study_group} {fullname} {suffix}"
... var_total = 10
... variant = int(sha3_256(msg.encode('utf-8')).hexdigest(), 16) % var_total + 1
... print(variant)
...
8
```

Рис. 1 - Расчет индивидуального варианта задания.

Тип приложения: Вариант №8 — Поисковая система.

Основные вызовы:

- Индексация больших коллекций документов.
- Обработка сложных поисковых запросов с низкой задержкой.
- Кэширование часто запрашиваемых результатов для снижения нагрузки на БД.

1. АРХИТЕКТУРА СИСТЕМЫ

1.1. Архитектура системы

Спроектированная система представляет собой микросервисную архитектуру, построенную по трехзвенной модели.

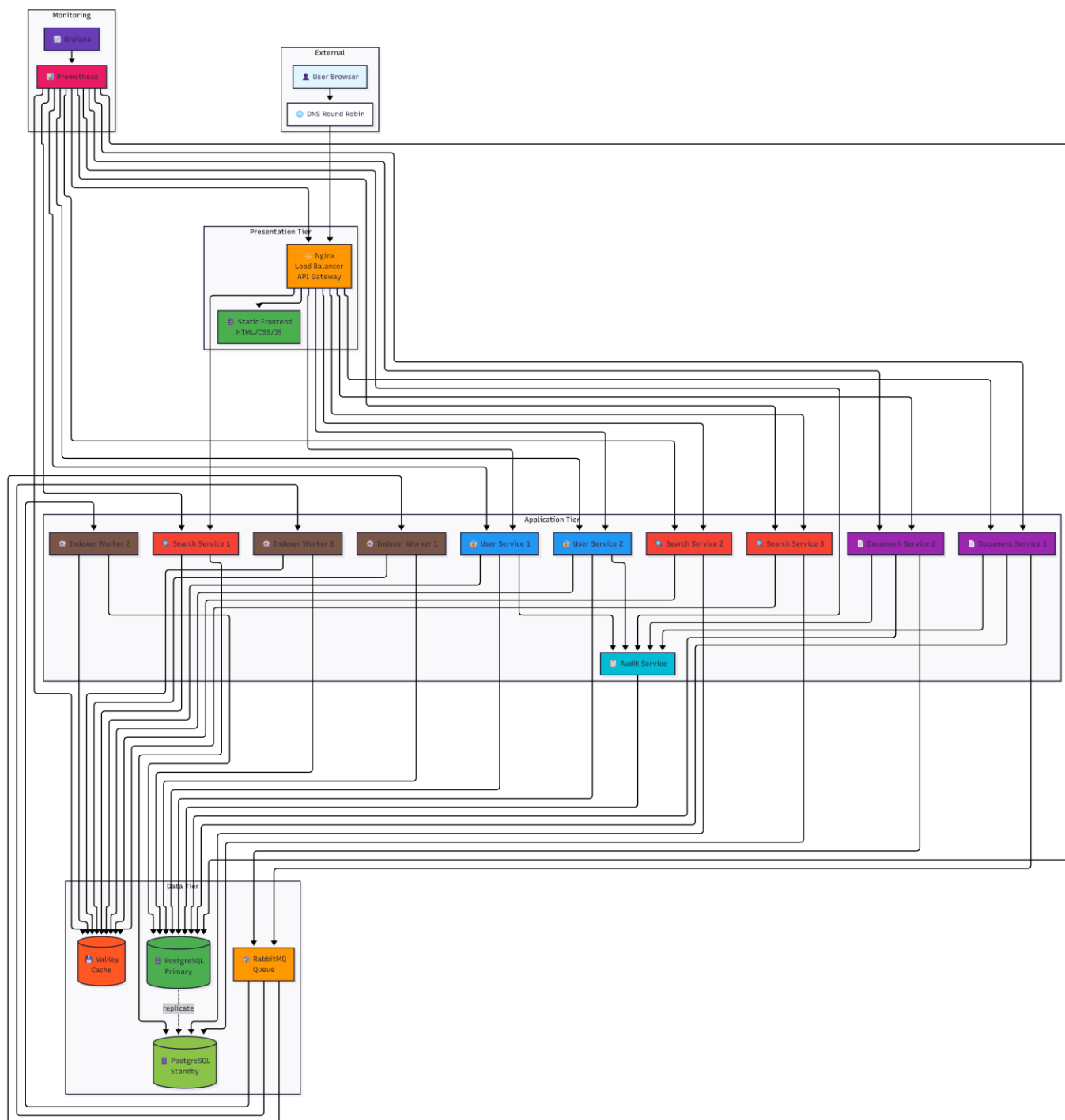


Рис. 2 - Высокоуровневая архитектура поисковой системы.

Ключевые компоненты:

1. **Уровень представления (Frontend):** Nginx выступает в роли API Gateway, балансировщика нагрузки, кэширующего прокси и сервера для статического контента.
2. **Уровень приложения (Backend):** Реализован на Python (FastAPI) в виде 5 независимых микросервисов:
 - User Service: Управление пользователями и аутентификация.
 - Document Service: CRUD-операции с документами.
 - Search Service: Обработка поисковых запросов.
 - Indexer Service: Асинхронная индексация документов.
 - Audit Service: Логирование всех событий по паттерну Event Sourcing.
3. **Уровень данных (Data):**
 - PostgreSQL: Отказоустойчивый кластер с автоматическим failover (pg_auto_failover).
 - RabbitMQ: Брокер сообщений для асинхронных задач.
 - Valkey (Redis-fork): Распределенный кэш для сессий и результатов поиска.

2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1. Трехзвенная архитектура и микросервисы

Система была полностью контейнеризована с помощью Docker. Файл `docker-compose.yml` описывает запуск и взаимодействие 15+ контейнеров, реализующих трехзвенную архитектуру.

```
PS C:\Users\basil> docker --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES                                STATUS                                PORTS
search-indexer-worker               Up 2 hours                            8000/tcp
search-user-service-v2              Up 3 hours (healthy)                 8000/tcp
search-user-service-v1              Up 3 hours (healthy)                 8000/tcp
search-grafana                       Up 3 hours                           0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
search-document-service              Up 4 hours (healthy)                 8000/tcp
search-rabbitmq                      Up 4 hours (healthy)                 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp,
0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
search-nginx-exporter                Up 4 hours                           0.0.0.0:9113->9113/tcp, [::]:9113->9113/tcp
search-nginx-canary                  Up 4 hours (unhealthy)               0.0.0.0:80->80/tcp, [::]:80->80/tcp
search-audit-service                 Up 4 hours (unhealthy)               8000/tcp
search-valkey                        Up 4 hours (healthy)                 0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
search-prometheus                    Up 2 hours                           0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
search-search-service                Up 4 hours (healthy)                 8000/tcp
pgauto-node2                         Up 4 hours (healthy)                 0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
pgauto-node1                         Up 2 hours (healthy)                 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
pgauto-monitor                       Up 4 hours (healthy)                 0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp
```

Рис. 3 - Список контейнеров системы, запущенных через Docker Compose.

2.2. Конфигурация Nginx: балансировка и кэширование

Nginx настроен как обратный прокси, который распределяет запросы между экземплярами микросервисов, используя upstream блоки. Для ускорения ответов на поисковые запросы используется встроенный механизм кэширования `proxy_cache`.

```

PS C:\Users\basil> curl -I "http://localhost/api/search?q=docker"
HTTP/1.1 200 OK
Server: nginx/1.29.2
Date: Mon, 27 Oct 2025 19:17:42 GMT
Content-Type: application/json
Content-Length: 535
Connection: keep-alive
Vary: Accept-Encoding
X-Cache-Status: MISS

PS C:\Users\basil> curl -I "http://localhost/api/search?q=docker"
HTTP/1.1 200 OK
Server: nginx/1.29.2
Date: Mon, 27 Oct 2025 19:18:05 GMT
Content-Type: application/json
Content-Length: 535
Connection: keep-alive
Vary: Accept-Encoding
X-Cache-Status: HIT

```

Рис. 4 - Демонстрация работы кэша Nginx (первый запрос - MISS, второй - HIT).

2.3. Настройка базы данных с репликацией

Для обеспечения высокой доступности базы данных был развернут кластер PostgreSQL с использованием `pg_auto_failover`. Кластер состоит из трех узлов: одного монитора, одной `primary`-ноды (для записи) и одной `standby`-ноды (для чтения и горячего резерва). В случае отказа `primary`-ноды, `standby` автоматически становится новой `primary`.

```

PS C:\Users\basil> docker exec -it pgauto-monitor pg_autoctl show state

```

Name	Node	Host:Port	TLI: LSN	Connection	Current State	Assigned State
node_1	1	pgauto-node1:5432	2: 0/3146AF8	read-only	secondary	secondary
node_2	2	pgauto-node2:5432	2: 0/3146AF8	read-write	primary	primary

Рис. 5 - Исходное состояние кластера `pg_auto_failover` (одна `primary`, одна `secondary`).

Для демонстрации отказоустойчивости был проведен тест, имитирующий сбой `primary`-ноды.

```
PS C:\Users\basil> docker exec pgauto-monitor pg_autoctl show state
```

Name	Node	Host:Port	TLI: LSN	Connection	Current State	Assigned State
node_1	1	pgauto-node1:5432	7: 0/D000530	read-write	primary	primary
node_2	2	pgauto-node2:5432	7: 0/D0005A0	read-only	secondary	secondary

Рис. 6 - Результат теста: standby-нода успешно стала новой primary-нодой после сбоя.

2.4. Асинхронная обработка

Тяжелая и длительная операция индексации документов вынесена в асинхронную обработку с помощью очереди сообщений RabbitMQ. Это позволяет мгновенно отвечать пользователю при создании документа, не дожидаясь завершения индексации.

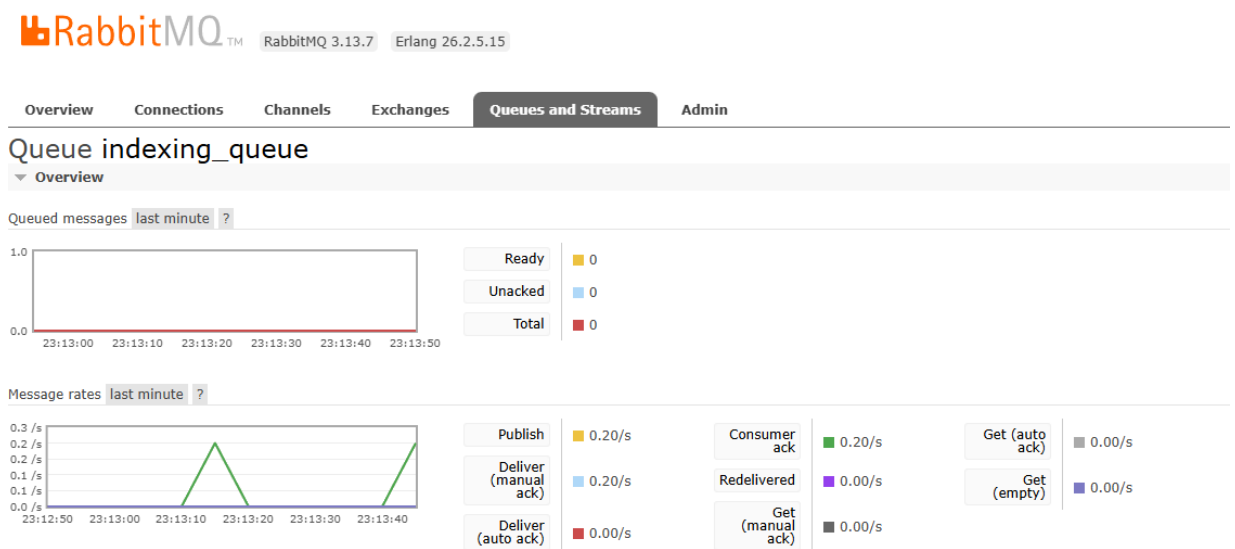


Рис. 7 - Панель управления RabbitMQ, демонстрирующая обработку задач в очереди indexing_queue.

2.5. Симуляция DNS-балансировки

Для демонстрации принципа DNS-балансировки была создана симуляция с помощью dnsmasq, эмулирующая три разных дата-центра. dnsmasq настроен на возврат IP-адресов в режиме Round Robin.

```

PS C:\Users\basil\Documents\GitHub\highload-systems> dig '@127.0.0.1' -p 8053 search.lab A +short
127.0.0.1
127.0.0.3
127.0.0.2
PS C:\Users\basil\Documents\GitHub\highload-systems> dig '@127.0.0.1' -p 8053 search.lab A +short
127.0.0.3
127.0.0.2
127.0.0.1
PS C:\Users\basil\Documents\GitHub\highload-systems> dig '@127.0.0.1' -p 8053 search.lab A +short
127.0.0.2
127.0.0.1
127.0.0.3
PS C:\Users\basil\Documents\GitHub\highload-systems> dig '@127.0.0.1' -p 8053 search.lab A +short
127.0.0.1
127.0.0.3
127.0.0.2
PS C:\Users\basil\Documents\GitHub\highload-systems> dig '@127.0.0.1' -p 8053 search.lab A +short
127.0.0.3
127.0.0.2
127.0.0.1
PS C:\Users\basil\Documents\GitHub\highload-systems> dig '@127.0.0.1' -p 8053 search.lab A +short
127.0.0.2
127.0.0.1
127.0.0.3
PS C:\Users\basil\Documents\GitHub\highload-systems> |

```

Рис. 8 - Результат теста DNS Round Robin, показывающий равномерное распределение нагрузки.

2.6. Мониторинг и логирование

Система оснащена стеком мониторинга Prometheus + Grafana. Микросервисы экспортируют метрики (RPS, latency, ошибки), которые собираются Prometheus и визуализируются в Grafana.

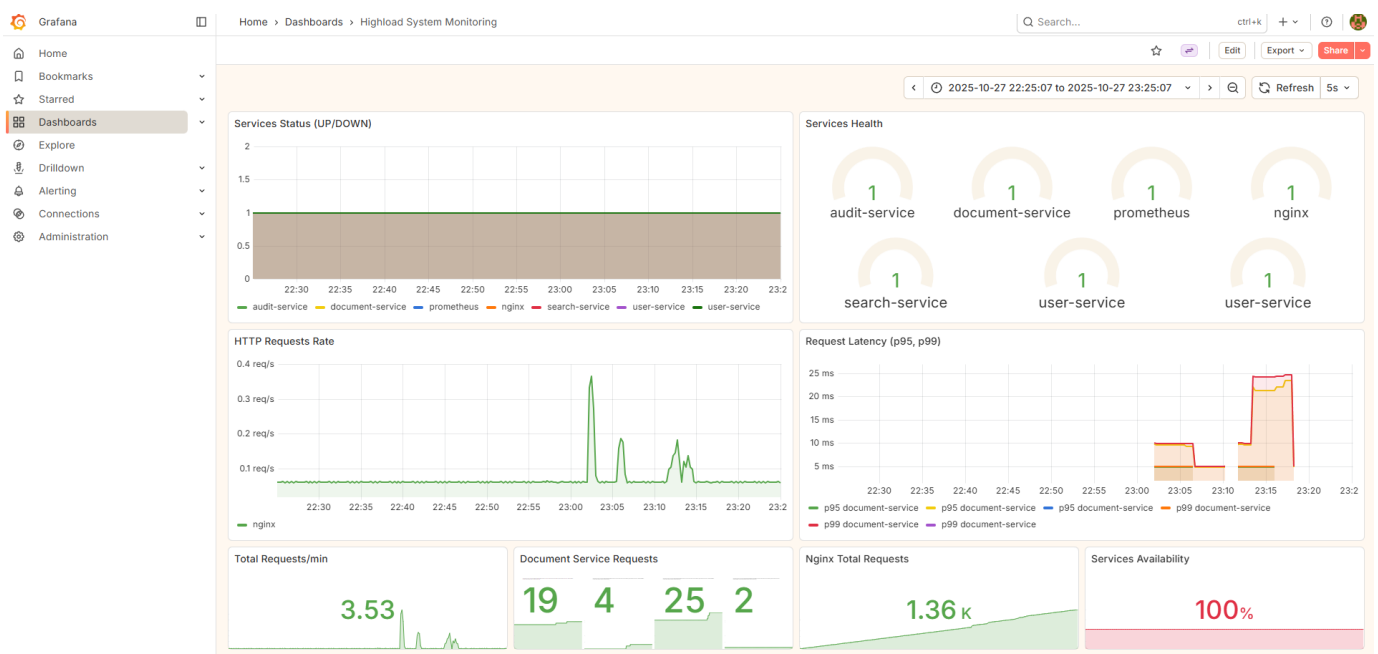


Рис. 9 - Дашборд в Grafana, отображающий ключевые метрики системы под нагрузкой.



Prometheus

Query

Alerts

Status > Target health



Select scrape pool

Filter by target health

Filter by endpoint or labels



audit-service

1 / 1 up

Endpoint

Labels

Last scrape

State

<http://audit-service:8000/metrics>

instance="audit-service:8000"

job="audit-service"

service="audit-service"

2.42s ago

2ms

UP

document-service

1 / 1 up

Endpoint

Labels

Last scrape

State

<http://document-service:8000/metrics>

instance="document-service:8000"

job="document-service"

service="document-service"

13.67s ago

2ms

UP

nginx

1 / 1 up

Endpoint

Labels

Last scrape

State

<http://nginx-exporter:9113/metrics>

instance="nginx-exporter:9113"

job="nginx"

service="nginx"

6.712s ago

2ms

UP

prometheus

1 / 1 up

Endpoint

Labels

Last scrape

State

<http://localhost:9090/metrics>

instance="localhost:9090"

job="prometheus"

15.374s ago

5ms

UP

search-service

1 / 1 up

Endpoint

Labels

Last scrape

State

<http://search-service:8000/metrics>

instance="search-service:8000"

job="search-service"

service="search-service"

7.53s ago

3ms

UP

user-service

2 / 2 up

Endpoint

Labels

Last scrape

State

<http://user-service-v1:8000/metrics>

instance="user-service-v1:8000"

job="user-service"

service="user-service"

10.167s ago

2ms

UP

<http://user-service-v2:8000/metrics>

instance="user-service-v2:8000"

job="user-service"

service="user-service"

11.176s ago

2ms

UP

Рис. 10 - Статус сбора метрик в Prometheus, все цели в состоянии UP.

2.7. Резервное копирование и восстановление

Реализован механизм автоматического резервного копирования с помощью `pg_dump` и скриптов для восстановления. Проведен тест, подтверждающий целостность данных после цикла бэкап-восстановление.

```
PS C:\Users\basil\Documents\GitHub\highload-systems> .\scripts\backup.ps1
```

Скрипт резервного копирования PostgreSQL

Каталог для копий: .\backups\2025-10-28_02-07-35

Начато копирование базы app_db...

Готово: app_db (408.74 KB)

Резервное копирование завершено

Каталог с копиями: .\backups\2025-10-28_02-07-35

Созданные файлы:

Name	Length	LastWriteTime
app_db.sql	418546	28.10.2025 2:07:36

Рис. 11 – Создание резервной копии БД

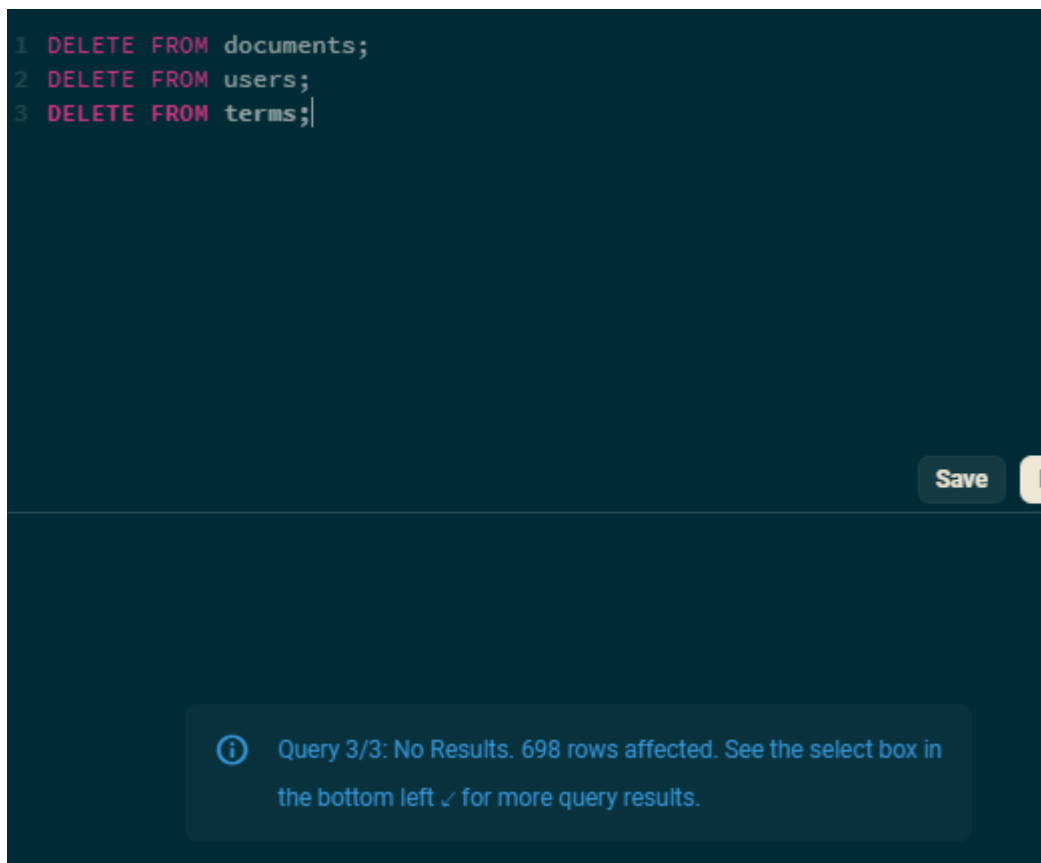


Рис. 12 – Полная очистка БД.

```

PS C:\Users\basil\Documents\GitHub\highload-systems> .\scripts\restore.ps1

cmdlet restore.ps1 at command pipeline position 1
Supply values for the following parameters:
BackupPath: .\backups\2025-10-28_02-07-35

```

Скрипт восстановления PostgreSQL

```

Источник: .\backups\2025-10-28_02-07-35
Целевой контейнер: pgauto-node1

⚠ ВНИМАНИЕ: база app_db будет пересоздана
Введите 'yes' для продолжения: yes
🔧 Восстановление базы app_db...
pg_terminate_backend
-----
 t
 t
 t
 t
(4 rows)

DROP DATABASE
CREATE DATABASE
SET
SET
SET
SET
SET
set_config
-----
(1 row)

SET
SET
SET

```

Рис. 13 Восстановление БД из резервной копии.

```
1 SELECT COUNT(*) FROM documents;
2 SELECT COUNT(*) FROM users;
3 SELECT COUNT(*) FROM terms;
```

	count
1	698

Рис. 14 - Успешное завершение теста резервного копирования и восстановления.

2.8. Event Sourcing и партиционирование

Для аудита всех действий реализован паттерн Event Sourcing. Все изменения сохраняются в виде событий в таблицу events, которая партиционирована по диапазону дат (помесячно) для обеспечения высокой производительности при работе с большими объемами данных.

1	SELECT					
2	child.relname as partition_name,					
3	pg_size_pretty(pg_relation_size(child.oid)) as partition_size,					
4	(SELECT count(*) FROM events WHERE tableoid = child.oid) as row_count,					
5	(SELECT MIN(timestamp) FROM events WHERE tableoid = child.oid) as first_event,					
6	(SELECT MAX(timestamp) FROM events WHERE tableoid = child.oid) as last_event					
7	FROM pg_inherits					
8	JOIN pg_class parent ON pg_inherits.inhparent = parent.oid					
9	JOIN pg_class child ON pg_inherits.inhrelid = child.oid					
10	WHERE parent.relname = 'events'					
					Save	Run
	partition_name ▲	partition_size ▲	row_count ▲	first_event ▲	last_event ▲	
1	events_2024_10	8192 bytes	2	2024-10-15 00:00:00+00	2024-10-15 00:00:00+00	
2	events_2024_11	8192 bytes	2	2024-11-20 00:00:00+00	2024-11-20 00:00:00+00	
3	events_2024_12	0 bytes	0	(NULL)	(NULL)	
4	events_2025_01	8192 bytes	2	2025-01-10 00:00:00+00	2025-01-10 00:00:00+00	
5	events_2025_02	0 bytes	0	(NULL)	(NULL)	
6	events_2025_03	0 bytes	0	(NULL)	(NULL)	
7	events_2025_04	0 bytes	0	(NULL)	(NULL)	
8	events_2025_05	0 bytes	0	(NULL)	(NULL)	
9	events_2025_06	8192 bytes	2	2025-06-15 00:00:00+00	2025-06-15 00:00:00+00	
10	events_2025_07	0 bytes	0	(NULL)	(NULL)	
11	events_2025_08	0 bytes	0	(NULL)	(NULL)	
12	events_2025_09	0 bytes	0	(NULL)	(NULL)	
13	events_2025_10	0 bytes	0	(NULL)	(NULL)	
14	events_2025_11	0 bytes	0	(NULL)	(NULL)	
15	events_2025_12	8192 bytes	2	2025-12-20 00:00:00+00	2025-12-20 00:00:00+00	
16	events_2026_01	0 bytes	0	(NULL)	(NULL)	
17	events_2026_02	0 bytes	0	(NULL)	(NULL)	
18	events_2026_03	0 bytes	0	(NULL)	(NULL)	
19	events_2026_04	0 bytes	0	(NULL)	(NULL)	

Рис. 15 - Распределение данных по партициям

<pre> 1 EXPLAIN (ANALYZE, BUFFERS, VERBOSE) 2 SELECT * FROM events 3 WHERE timestamp >= '2025-01-01'::timestampz 4 AND timestamp < '2025-02-01'::timestampz; </pre>		Save
QUERY PLAN		
1	Append (cost=0.00..12.56 rows=1 width=432) (actual time=0.007..0.008 rows=2 loops=1)	
2	Buffers: shared hit=1	
3	-> Seq Scan on public.events_2025_01 (cost=0.00..12.55 rows=1 width=432) (actual time=0.007..0.007 rows=2 loops=1)	
4	Output: events_2025_01.id, events_2025_01.aggregate_type, events_2025_01.aggregate_id, events_2025_01.event_type, events_2025_01.event_data, events_2025_01.user_id, ev...	
5	Filter: ((events_2025_01.timestamp' >= '2025-01-01 00:00:00+00':timestamp with time zone) AND (events_2025_01.timestamp' < '2025-02-01 00:00:00+00':timestamp with tim...	
6	Buffers: shared hit=1	
7	Planning Time: 6.644 ms	
8	Execution Time: 0.033 ms	

Рис. 16 - EXPLAIN с Partition Pruning

2.9. Канареечное развертывание (Canary Deployment)

Для безопасного выката новых версий реализована стратегия Canary Deployment. Nginx с помощью директивы `split_clients` направляет 10% трафика на новую версию `user-service-v2` и 90% на стабильную `v1`.

```

PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-1" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":100,"exceeded":false}
PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-2" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":100,"exceeded":false}
PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-5" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":100,"exceeded":false}
PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-10" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":100,"exceeded":false}
PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-25" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":200,"exceeded":false,"version":"2.0.0"}
PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-50" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":100,"exceeded":false}
PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-100" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":100,"exceeded":false}
PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-200" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":100,"exceeded":false}
PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-300" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":100,"exceeded":false}
PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-400" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":100,"exceeded":false}
PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-500" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":100,"exceeded":false}
PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-600" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":100,"exceeded":false}
PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-700" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":100,"exceeded":false}
PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-800" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":200,"exceeded":false,"version":"2.0.0"}
PS C:\Users\basil\Documents\GitHub\highload-systems> curl -H "User-Agent: TestClient-1000" http://localhost/api/users/1/quota
{"user_id":1,"quota_per_minute":200,"exceeded":false,"version":"2.0.0"}
PS C:\Users\basil\Documents\GitHub\highload-systems> |

```

Рис. 17 - Результаты теста канареечного развертывания, показывающие распределение трафика между v1 и v2.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была успешно спроектирована и реализована архитектура высоконагруженной поисковой системы. Были применены ключевые принципы и технологии, такие как микросервисная архитектура, горизонтальное масштабирование, асинхронная обработка, отказоустойчивая репликация баз данных, многоуровневое кэширование, канареечное развертывание и комплексный мониторинг.

Созданная система демонстрирует высокую степень надежности, масштабируемости и наблюдаемости, что полностью соответствует целям лабораторной работы. Практическая реализация всех компонентов и автоматизированные тесты подтверждают корректность выбранных архитектурных решений.

СПИСОК ЛИТЕРАТУРЫ

Бунин О., Лапшин М., Осипов К., Машуков К. *Учебник по высоким нагрузкам*. М.: Издательство Олега Бунина, 2018. 55 с